AMRITA VISHWA VIDYAPEETHAM

School of Computing — Department of Computer Science and Engineering

LegalAI

An Intelligent Criminal Legal Assistance and Analytics System

Developer Implementation Plan — Internal Working Document

Course Code 23CSE399 — Final Year Project

Project Guide Dr. Sabarish B A, Assistant Professor, Dept. of CSE

Development Team

Anantha Ram G S	CB.SC.U4CSE23408	Backend & AI Pipeline Lead
Rama Roshinee S V	CB.SC.U4CSE23645	Backend Developer
Ravindran G	CB.SC.U4CSE23647	Frontend Developer
Gugan S S	CB.SC.U4CSE23416	Frontend Developer

Academic Year 2024–2025 — Document Version 1.1

Contents

1 Purpose of This Document **2**

2 Confirmed System Architecture **2**
2.1 Layer 1 — Frontend (Next.js + TypeScript) . . . 2
2.2 Layer 2 — Backend (FastAPI + Python) . . . 2
2.3 Layer 3 — AI and Intelligence Layer . . . 2
2.4 Layer 4 — Data Layer . . . 3

3 Confirmed Technology Stack **3**

4 Complete System Flow **4**
4.1 Flow 1 — Document Ingestion Pipeline . . . 4
4.2 Flow 2 — Professional Mode Query (Lawyer) . . . 4
4.3 Flow 3 — Plain Language Mode Query (Public) . . . 5
4.4 Flow 4 — Analytics Dashboard . . . 5

5 Pages and API Endpoints **5**
5.1 Frontend Pages . . . 5
5.2 Backend API Endpoints . . . 6

6 Phase 0 — Pre-Implementation Setup **7**
6.0.1 Step 0.1 — Repository and Project Structure — Ananthu . . . 8
6.0.2 Step 0.2 — Environment and Interface Freeze — All Members . . . 8
6.0.3 Step 0.3 — PostgreSQL Schema Initialisation — Roshinee . . . 9
6.0.4 Step 0.4 — Dataset Collection and Preprocessing — Distributed . . . 9
6.0.5 Step 0.5 — InLegalBERT Data Preparation and Fine-tuning — Ananthu with Team Support . . . 10
6.0.6 Step 0.6 — Initial Corpus Ingestion — Ananthu + Roshinee . . . 10

7 Implementation Plan — Phases and Tasks **10**
7.1 Phase 1 — Parallel Core Implementation . . . 11
7.1.1 **Ananthu** [AI, Retrieval + Query Pipeline] . . . 11
7.1.2 **Roshinee** [Backend, Data + Application Services] . . . 12
7.2 Phase 2 — Frontend: Professional Mode . . . 12
7.2.1 **Ravindran** [Frontend Developer — Professional Mode] . . . 12
7.3 Phase 3 — Frontend: Plain Language Mode and Admin . . . 13
7.3.1 **Gugan** [Frontend Developer — Public Mode, Admin + Shared UX] . . . 13
7.4 Phase 4 — Integration, Evaluation, and Deployment . . . 14
7.4.1 **Ananthu** [AI Evaluation + Performance] . . . 14

8 Task Dependencies and Build Order **14**

9 Future Work **16**

Purpose of This Document

This document is the primary working reference for the LegalAI development team. It defines the confirmed architecture, the complete technology stack, the system flow, the full set of pages and API endpoints to be built, and a detailed implementation plan broken into phases and clearly owned work packages.

Every developer on this team should read this document in full before writing application code. The assignments in Sections 6 and 7 are designed to make ownership unambiguous while allowing independent work to proceed in parallel wherever possible. A dependency is considered blocking only when a task genuinely cannot proceed without it; frontend and backend work may otherwise use frozen API contracts and mock implementations until live integration is available.

This is a living document. As the project evolves, decisions made here may be updated. Architectural or API-contract changes must be discussed with the team lead before implementation begins.

Confirmed System Architecture

The system is structured into four layers. Each layer has a well-defined responsibility and communicates with other parts of the system through explicit service interfaces and API contracts.

Layer 1 — Frontend (Next.js + TypeScript)

A single Next.js application provides three interfaces: Professional Mode for authenticated lawyer users, Plain Language Mode for the general public, and an authenticated Admin interface. Public Mode is accessible without login. Professional and Admin routes are role-protected. All frontend pages communicate with the backend exclusively through REST API calls to FastAPI.

Layer 2 — Backend (FastAPI + Python)

Three internal services operate behind a single FastAPI application:

- **Ingestion Service** — accepts administrator-supplied corpus PDFs, runs parsing and OCR, chunks text, generates BGE-M3 embeddings, writes vector data to ChromaDB, and writes structured metadata to PostgreSQL.
- **Query Service** — receives text or temporary user-uploaded documents, orchestrates the NLP and RAG pipeline, calls the configured Gemini Flash model, and returns structured responses. User documents submitted for analysis are not automatically added to the global corpus.
- **Analytics Service** — runs structured queries over PostgreSQL and returns data for dashboard visualisation.

JWT-based authentication is used for protected routes. The backend determines the role from the authenticated user record and embeds it in the JWT. The token is stored in a secure HttpOnly cookie and is not exposed to client-side JavaScript.

Layer 3 — AI and Intelligence Layer

Two subsystems operate in sequence where applicable:

Subsystem A — InLegalBERT (Classification and Extraction): Runs on case descriptions and legal documents to extract structured legal entities and to predict likely IPC/BNS sections. The model-facing labels are mapped into a LegalAI canonical entity schema before being used by downstream services. Predicted sections are treated as supplementary retrieval signals rather than as definitive

legal conclusions.

Subsystem B — RAG Pipeline (Retrieval and Generation): LlamaIndex orchestrates hybrid retrieval — semantic search over ChromaDB using BGE-M3 embeddings combined with BM25 keyword search for exact legal terminology and section matches. The top-20 candidates are fused and deduplicated, then reranked by a BGE reranker to produce the top-5. Retrieved metadata is used directly to construct citation objects; the LLM generates the explanatory analysis but does not invent case metadata.

Layer 4 — Data Layer

- **ChromaDB** — stores BGE-M3 vector embeddings of corpus chunks with retrieval metadata including document ID, document type, court information, year, source URL, cited IPC/BNS sections, and chunk provenance.
- **PostgreSQL** — stores users, structured document metadata, chunk metadata, query history, ingestion jobs, analytics data, NCRB statistics, and IPC-to-BNS mapping data.

Confirmed Technology Stack

The following stack is locked at the architectural level. Version-specific external services may be configured through environment variables to avoid coupling the system to a retired model identifier.

Component	Technology	Reason
Primary LLM	Google Gemini Flash family	Current Flash model selected through `GEMINI_MODEL`; long context and structured generation support
Local Dev LLM	Ollama + Mistral 7B	Zero API cost during development iteration; local fallback interface
Legal NLP Model	InLegalBERT (fine-tuned)	Pre-trained on Indian legal text; lightweight encoder
Embeddings	BGE-M3	Multilingual; strong semantic retrieval performance for legal English
Vector Store	ChromaDB	Metadata filtering; developer-friendly; appropriate project scale
Retrieval	LlamaIndex (hybrid)	Hybrid semantic + BM25 orchestration support
Reranker	BGE Reranker	Cross-encoder reranking; runs locally on M5
Backend	FastAPI + Python	Native AI stack integration; async support
Relational DB	PostgreSQL	Structured metadata, jobs, history, and analytics queries
Analytics	Pandas + SQLAlchemy	Standard Python data stack; PostgreSQL integration
PDF Parsing	PyMuPDF (fitz)	Fast; handles complex Indian court PDF layouts
OCR	Tesseract (pytesseract)	Scanned Indian court document support

Component	Technology	Reason
Frontend	Next.js + TypeScript	Team React familiarity; SSR; type safety
Visualisation	Recharts	React-native charting; sufficient for dashboard needs
Auth	JWT (PyJWT)	Stateless role-based access control using secure HttpOnly cookies

Complete System Flow

Flow 1 — Document Ingestion Pipeline

This pipeline is used only for trusted corpus ingestion during initial setup and for documents deliberately added later through the Admin interface or standalone scripts.

1. Administrator or ingestion script supplies a corpus PDF and document metadata.
2. Shared document parser uses PyMuPDF for text extraction. If extracted text is empty or below threshold, Tesseract OCR is applied.
3. Text cleaning and normalisation removes repeated headers/footers where possible, fixes encoding issues, and normalises whitespace.
4. Structure-aware chunking splits by legal structure before falling back to token-count splitting. Target chunk size: 400–600 tokens with 50-token overlap.
5. Each chunk is embedded using BGE-M3 in configurable batches.
6. Chunks, embeddings, metadata, and provenance are written to ChromaDB.
7. Document and chunk metadata are written to PostgreSQL.
8. The persistent ingestion job record is updated with completion status, progress, chunk count, and any error information.

Flow 2 — Professional Mode Query (Lawyer)

1. Lawyer uploads a PDF or types a case description in the Professional Mode UI.
2. `POST /query/professional` receives exactly one of text input or a temporary PDF upload. The PDF is parsed for analysis but is not automatically inserted into the corpus.
3. InLegalBERT NER extracts structured entities. Model labels are mapped into the LegalAI canonical entity schema.
4. InLegalBERT section classification predicts likely IPC/BNS sections with confidence scores.
5. A legal-regime resolution step uses explicit statute references and available dates to distinguish the applicable IPC/CrPC or BNS/BNSS context and to expose mappings separately where useful.
6. The retrieval query is enriched while preserving the original user query as the primary signal. Explicit section references receive higher weight than model-predicted sections.
7. LlamaIndex hybrid retrieval executes semantic search plus BM25 keyword search. Top-20 candidates are fused and deduplicated.

8. BGE Reranker scores the candidates and returns the top-5.
9. The backend constructs precedent citation objects directly from retrieved metadata, including document ID, case name, court, year, relevance score, source URL, excerpt, and available page/paragraph provenance.
10. The configured Gemini Flash model receives the retrieved evidence and generates the structured case brief. Unsupported claims are explicitly avoided.
11. Response returned to frontend: identified sections with confidence/context, top-5 precedent citations, and structured case brief.

Flow 3 — Plain Language Mode Query (Public)

1. User types a question or uploads a legal document in the Public Mode UI. No login is required.
2. If a document is uploaded, the shared document parser extracts text temporarily; the document is not added to the corpus.
3. InLegalBERT NER runs where useful to identify legal references and contextual entities.
4. Hybrid retrieval returns top-20 candidates and the reranker selects top-5.
5. The prompt is assembled with Plain Language Mode instructions: avoid unnecessary legal jargon, explain citations in accessible language, and state uncertainty when retrieved evidence is insufficient.
6. The configured Gemini Flash model generates the response.
7. Response returned: plain-English explanation, simplified sections, practical next-step information, and simplified source references.

Flow 4 — Analytics Dashboard

1. Lawyer opens the Analytics tab in Professional Mode.
2. Frontend requests analytics data from FastAPI Analytics Service.
3. SQLAlchemy queries execute over PostgreSQL for each visualisation: offence frequency by section and year, outcome distribution by section and court level, section co-occurrence, IPC-to-BNS transition mapping, and NCRB crime statistics trends.
4. JSON data returned using frozen response contracts.
5. Recharts renders each visualisation in the dashboard.

Pages and API Endpoints

Frontend Pages

Route	Owner	Description
`/login`	Ravindran	Login page for protected roles. Email and password only; the backend derives the role from the user account. JWT is stored by the server in a secure HttpOnly cookie.
`/professional`	Ravindran	Professional Mode home. Quick-access cards to Upload, Query, Analytics, History.
`/professional/upload`	Ravindran	Private PDF analysis interface. Sends the document directly to `/query/professional`; it does not add the document to the global corpus.
`/professional/query`	Ravindran	Text query interface for lawyers. Same output structure as upload page.
`/professional/analytics`	Ravindran	Analytics dashboard. Tabbed: Offense Trends, Outcome Distribution, Section Co-occurrence, IPC-BNS Map, NCRB Stats.
`/professional/history`	Ravindran	Past authenticated professional queries and their outputs.
`/public`	Gugan	Plain Language Mode home. Publicly accessible, simple landing, prominent query input, upload button, zero legal jargon.
`/public/ask`	Gugan	Main public interaction page. Chat-style interface, text or temporary PDF upload, plain-English response, session follow-ups.
`/admin`	Gugan	System status panel. Document corpus stats, ChromaDB status, PostgreSQL status, last ingestion date.
`/admin/ingest`	Gugan	Corpus ingestion interface for administrators. Upload documents intentionally into the shared corpus and display persistent ingestion-job progress.

Backend API Endpoints

Method	Endpoint	Owner	Description
POST	`/auth/login`	Roshinee	Validates credentials, derives account role, sets JWT in secure HttpOnly cookie
POST	`/auth/logout`	Roshinee	Clears the authentication cookie; no in-memory JWT blacklist required for Phase 1
POST	`/ingest/upload`	Roshinee	Admin-only corpus ingestion; accepts PDF + metadata and returns persistent job ID
GET	`/ingest/status/{job_id}`	Roshinee	Returns ingestion job stage, progress, result, or error
GET	`/ingest/corpus/stats`	Roshinee	Returns total documents, last updated, breakdown by type and service status
POST	`/query/professional`	Ananthu	Accepts text or temporary PDF; returns sections + citations + structured case brief
POST	`/query/public`	Ananthu	Public endpoint accepting text or temporary PDF; returns plain-language response
GET	`/query/history`	Roshinee	Returns authenticated user's professional query history
GET	`/analytics/offense-frequency`	Roshinee	Returns section frequency counts by year
GET	`/analytics/outcome-distribution`	Roshinee	Returns normalized outcome data by section and court
GET	`/analytics/section-cooccurrence`	Roshinee	Returns section co-occurrence matrix data
GET	`/analytics/ipc-bns-map`	Roshinee	Returns IPC-to-BNS mapping table
GET	`/analytics/ncrb-stats`	Roshinee	Returns NCRB crime statistics by year and category

Phase 0 — Pre-Implementation Setup

Lead: Ananthu — Shared preparation before and alongside Phase 1

Phase 0 is divided into a short blocking foundation and parallel data/model preparation. Ananthu owns the repository and project structure. The remaining setup and data preparation work is distributed so that application development does not wait for the complete dataset or final fine-tuned models. A small development corpus and frozen interface contracts are sufficient to begin Phase 1.

Step 0.1 — Repository and Project Structure — Ananthu

Create the GitHub organisation or repository and initialise the following directory structure:

```
legalai/
 backend/
    app/
       api/           # FastAPI route files
       services/      # Shared application services
       models/        # Pydantic request/response models
       db/            # PostgreSQL and ChromaDB connection
       nlp/           # InLegalBERT wrappers
       rag/           # Retrieval and generation pipeline
       core/          # Config, auth, middleware
    scripts/           # Data loading, ingestion, evaluation scripts
    requirements.txt
    main.py
 frontend/
    app/               # Next.js app directory
    components/        # Shared UI components
    lib/               # API client, auth helpers, shared types
    package.json
 data/
    raw/               # Raw downloaded source documents
    processed/         # Cleaned text / intermediate data
    structured/        # NCRB CSVs, IPC-BNS mapping
 models/                # Fine-tuned model checkpoints
 notebooks/             # Jupyter notebooks for data/model work
 docs/                  # This document and architecture diagrams
```

Set up branch protection on `main`. All developers push to feature branches and open pull requests. No direct commits to `main`. Ananthu maintains architectural consistency, while routine implementation reviews may be handled within each member's ownership area.

Step 0.2 — Environment and Interface Freeze — All Members

Ananthu creates the shared `.env.example`; Roshinee verifies backend/database variables; Ravindran and Gugan verify frontend environment requirements. Freeze the first version of request/response contracts before parallel feature work begins.

```
GEMINI_API_KEY=
GEMINI_MODEL=gemini-3.7-flash
DATABASE_URL=postgresql://user:password@localhost:5432/legalai
CHROMA_PERSIST_DIR=./chroma_store
JWT_SECRET_KEY=
```

```
JWT_ALGORITHM=HS256
JWT_EXPIRY_MINUTES=60
INLEGALBERT_MODEL_PATH=./models/inlegalbert-finetuned
OLLAMA_BASE_URL=http://localhost:11434
```

The initial API contract freeze must define at minimum: authentication response, professional query response, public query response, ingestion status response, corpus statistics response, analytics responses, and the common error response. Frontend development may use mock JSON conforming to these contracts before live endpoints are complete.

Step 0.3 — PostgreSQL Schema Initialisation — Roshinee

Create the PostgreSQL database, migration setup, and initial schema. The minimum tables are:

documents — `id, doc_type, case_name, court_name, court_level, case_number, decision_date, year, case_disposition, conviction_status, appeal_result, source_url, file_path, checksum, ingested_at`

chunks — `id, document_id (FK), chunk_index, text, chroma_id, token_count, page_start, page_end, paragraph_start, paragraph_end, section_heading`

document_sections — `id, document_id (FK), statute, section`

query_history — `id, user_id (nullable FK), session_id (nullable), mode, query_text, response_json, generation_backend, created_at`

users — `id, email, hashed_password, role (lawyer/admin), created_at`

ingestion_jobs — `id, document_id (nullable FK), status, progress, error_message, created_at, started_at, completed_at`

ipc_bns_mapping — `ipc_section, ipc_description, bns_section, bns_description, change_type`

ncrb_stats — `year, state, offense_category, ipc_section, case_count, conviction_count, acquittal_count`

Step 0.4 — Dataset Collection and Preprocessing — Distributed

Dataset work begins with a small representative development set so the pipeline can be implemented immediately, while larger collection and preprocessing continue in parallel.

1. **Ananthu — Judgments and statutory corpus:** obtain a development set first, then scale toward 8,000–10,000 criminal judgments subject to source/API access. Collect IPC, BNS, CrPC, and BNSS statutory texts. Record source metadata and attribution requirements.
2. **Roshinee — Structured analytics data:** collect and standardise NCRB annual data and prepare the IPC-to-BNS mapping dataset for PostgreSQL loading.
3. **Ravindran — FIR/case-fact dataset support:** assist in collecting labelled or labelable FIR/case-fact records across offence categories. Blank templates alone are not sufficient for section-classification training; records must pair narrative text with applicable section labels where possible.
4. **Gugan — Corpus metadata and data-quality support:** assist with source tracking, document-type categorisation, duplicate checking, and metadata verification for the collected legal corpus.

Step 0.5 — InLegalBERT Data Preparation and Fine-tuning — Ananthu with Team Support

Ananthu owns model design, training, evaluation, and final checkpoint integration. Ravindran and Gugan assist with annotation/data cleaning where required; Roshinee supports structured section mappings used for label normalisation.

Task A — Multi-label IPC/BNS Section Classification: Input: FIR/case-fact narrative. Output: applicable section labels with confidence scores. Historical IPC labels and derived/current BNS mappings must be distinguished in the dataset.

Task B — Legal Named Entity Recognition: Input: legal document text. Output: model-supported legal entities. A mapping layer converts model labels into the LegalAI canonical schema. Entities such as accused role or offence description may require contextual extraction beyond direct NER labels.

The application wrappers must be implemented before the final checkpoints are available so the rest of Phase 1 can use a base/pre-existing model, heuristic stub, or test fixture. Final checkpoints are swapped in after held-out evaluation.

Step 0.6 — Initial Corpus Ingestion — Ananthu + Roshinee

After the ingestion pipeline is operational, Ananthu and Roshinee jointly run staged corpus ingestion. Begin with a small corpus for integration testing, then scale only after retrieval quality and metadata integrity are verified.

1. Parse and validate a small development corpus.
2. Chunk documents and preserve source/page/paragraph provenance where available.
3. Generate BGE-M3 embeddings in batches.
4. Ingest chunks and metadata into ChromaDB.
5. Populate PostgreSQL document/chunk metadata and normalized section records.
6. Load NCRB and IPC-BNS structured datasets.
7. Scale progressively from the development corpus to the full target corpus after successful integration checks.

Implementation Plan — Phases and Tasks

The implementation is organised as parallel workstreams rather than a single sequential chain. Each work package has one clear owner. Tasks marked as contract-dependent may begin with mock/stub implementations once the agreed interface is frozen; only live integration must wait for the real dependency.

Phase 1 Review — 60% Completion Target

The following components constitute the target demonstration for the Phase 1 panel review:

- Corpus ingestion pipeline end-to-end (trusted PDF → ChromaDB + PostgreSQL)
- InLegalBERT NER and section classifier integrated and working
- RAG pipeline: hybrid retrieval + reranker returning grounded results

- `/query/professional` returning sections + citations + structured case brief
- `/query/public` returning plain-language responses
- Authentication and role protection working for Professional/Admin routes
- Analytics endpoints returning real PostgreSQL data
- Professional Mode frontend: `/login`, `/professional`, `/professional/upload`, `/professional/query`, `/professional/analytics`
- Plain Language Mode home page: `/public`

Plain Language Mode full interaction, Admin UI completion, comprehensive evaluation, deployment, and final polish follow as the remaining work.

Phase 1 — Parallel Core Implementation

Prerequisite: Step 0.1 complete and the initial environment/API contracts from Step 0.2 frozen. Full dataset collection and final fine-tuning continue in parallel.

Ananthu [AI, Retrieval + Query Pipeline]

Task 1.A.1 — Shared Document Processing and Vector Services
Implement the reusable document-processing and vector components: `document_parser.py` for PyMuPDF extraction, OCR fallback, cleaning and page tracking; `chunker.py` for structure-aware 400–600 token chunks with 50-token overlap and provenance; `embedding.py` for cached/batched BGE-M3 embeddings; and `chroma.py` for collection setup, upsert, metadata filtering, and query helpers. These services are the single implementation used by both corpus ingestion and query-time document analysis.

Task 1.A.2 — Legal NLP Services
Implement `ner.py` and `classifier.py` behind stable interfaces. Long documents must be processed in segments and merged. Section predictions return confidence scores. Add the canonical entity mapping and a lightweight legal-regime resolver that distinguishes explicit/applicable IPC/CrPC and BNS/BNSS context. Development may start with a base model or stub before final fine-tuned checkpoints are ready.

Task 1.A.3 — Hybrid Retrieval and Reranking
In `backend/app/rag/pipeline.py`, configure Chroma semantic retrieval, BM25 retrieval, reciprocal-rank fusion, candidate deduplication, and BGE reranking. Retrieve top-20 and return top-5. Preserve original query text as the strongest signal; explicit section references may guide filters/expansion, while model-predicted sections remain supplementary.

Task 1.A.4 — LLM and Structured Generation Layer
In `backend/app/rag/llm.py`, configure the Gemini Flash model using `GEMINI_MODEL` from the environment. Implement professional/public prompt builders and `generate()` with rate-limit/error handling. Keep Ollama + Mistral 7B behind the same interface as a local development fallback. Record which generation backend was used for debugging/evaluation. Require generated claims to remain grounded in supplied evidence.

Task 1.A.5 — Professional and Public Query Orchestration
Implement `POST /query/professional` and `POST /query/public`. Professional input

accepts either text or one temporary PDF. The professional pipeline performs parsing → NER → classification → regime/context resolution → weighted query enrichment → hybrid retrieval → reranking → citation construction from metadata → structured generation. The public pipeline performs parsing where needed → light entity extraction → retrieval → reranking → plain-language generation. Neither endpoint automatically inserts user uploads into the global corpus.

Professional response schema includes `identified_sections`, `precedents`, and structured `case_brief`. Public response includes `explanation`, simplified `sections_cited`, and `sources`.

Roshinee [Backend, Data + Application Services]

Task 1.R.1 — FastAPI, Database and Authentication Foundation
Set up `backend/main.py`, CORS, router wiring, configuration, `/health`, SQLAlchemy async PostgreSQL connection, migrations, and ORM models. Implement JWT creation/validation, current-user and role dependencies, `POST /auth/login`, `POST /auth/logout`, and development seed users. Login accepts email/password only; role is loaded from PostgreSQL. Authentication uses a secure HttpOnly cookie. Logout clears the cookie.

Task 1.R.2 — Corpus Ingestion and Job Management
Implement the ingestion orchestration service and its API surface: `POST /ingest/upload`, `GET /ingest/status/{job_id}`, and `GET /ingest/corpus/stats`. This service calls Ananthu's shared parser, chunker, embedding, and vector-store interfaces rather than duplicating them. Store ingestion state persistently in the `ingestion_jobs` table with stages such as `QUEUED`, `EXTRACTING`, `CHUNKING`, `EMBEDDING`, `INDEXING`, `COMPLETED`, and `FAILED`. FastAPI `BackgroundTasks` is sufficient for Phase 1 execution while the persistent database record provides restart-safe status tracking.

Task 1.R.3 — Analytics, History and Structured Data Services
Implement the five analytics query functions/endpoints, query-history service/endpoint, and the NCRB and IPC-BNS load scripts. Analytics functions cover offence frequency, normalized outcome distribution, section co-occurrence, IPC-BNS mapping, and NCRB filtering. The query-history service exposes a save function for Ananthu's query orchestrators and returns the latest authenticated professional queries. Public anonymous conversations remain session-local unless optional persistence is added later.

Phase 2 — Frontend: Professional Mode

Prerequisite: API contracts frozen. Pages may be developed against mock data before live endpoints are available.

Ravindran [Frontend Developer — Professional Mode]

Task 2.V.1 — Professional Frontend Foundation
Initialise the Next.js + TypeScript project and implement the Professional Mode frontend foundation: `api.ts` with credentials enabled for HttpOnly-cookie authentication, shared TypeScript API types, `Navbar.tsx`, `CitationCard.tsx`, and `SectionBadge.tsx`. Do not expose JWT-reading helpers in client-side JavaScript. Coordinate with Gugan on the shared loading/error/upload components he owns.

Task 2.V.2 — Login and Protected Route Flow
Build `/login` with email and password fields only. On successful login, redirect according

to the backend-returned account role. Implement route protection for `/professional/*` and `/admin/*`; unauthenticated requests redirect to `/login`, and role mismatches redirect appropriately.

Task 2.V.3 — Professional Home, Upload and Text Query
Build `/professional`, `/professional/upload`, and `/professional/query`. The upload page sends the private PDF directly to `/query/professional`; it must not call the corpus-ingestion endpoint. Both upload and text-query pages render identified sections, retrieved precedent citations, and the structured case brief. UI implementation may use frozen mock responses before the live query endpoint is ready.

Task 2.V.4 — Professional Analytics Dashboard
Build the five-tab Recharts dashboard for Offense Trends, Outcome Distribution, Section Co-occurrence, IPC-BNS Map, and NCRB Statistics. Chart/table development begins with agreed mock JSON and switches to live endpoints when Roshinee's analytics APIs are available.

Task 2.V.5 — Query History
Build `/professional/history` using the frozen history contract, with truncated query text, mode/timestamp, expandable stored result, and client-side pagination.

Phase 3 — Frontend: Plain Language Mode and Admin

Prerequisite: API contracts frozen. Public/Admin UI can be built with mock responses in parallel with backend implementation.

Gugan [Frontend Developer — Public Mode, Admin + Shared UX]

Task 3.G.1 — Shared UX Components
Build reusable `LoadingSpinner.tsx`, `ErrorMessage.tsx`, `UploadDropzone.tsx`, `IngestionProgress.ts` and `ResponseCard.tsx`. Ravindran may consume these components but should not create duplicate implementations. `ResponseCard` must support visually distinct professional and public source presentation.

Task 3.G.2 — Public Home and Main Interaction
Build `/public` and `/public/ask`. Public Mode requires no login. The chat interface accepts text or a temporary PDF, keeps the current conversation locally for follow-up questions, renders simplified source references, and provides "Start New Conversation". No uploaded public document is added to the global corpus.

Task 3.G.3 — Admin Status and Corpus Ingestion
Build `/admin` and `/admin/ingest`. Admin routes require the admin role. The ingestion page supports multiple PDFs with metadata per file, calls `POST /ingest/upload`, and polls persistent `GET /ingest/status/{job_id}` results. The status page displays corpus counts, database/vector-store health, and last ingestion time.

Task 3.G.4 — Responsive and Plain-Language UX
Ensure Public Mode is fully usable from 375px width upward and the chat/input layout reflows correctly. Shared components should also prevent Professional/Admin pages from breaking on tablet widths. Public-facing errors and labels must avoid technical jargon.

Phase 4 — Integration, Evaluation, and Deployment

Prerequisite: live vertical integrations are available. Testing begins incrementally before this phase; this phase completes system-wide evaluation and deployment.

Ananthu [AI Evaluation + Performance]

Task 4.A.1 — RAG and NLP Evaluation
Build `evaluate_rag.py` and `evaluate_nlp.py`. For RAG, prepare approximately 100 queries with known relevant judgments and measure Precision@5, Recall@5, Recall@20, Mean Reciprocal Rank, and citation grounding rate. Compare three configurations: LLM-only, semantic retrieval without the full hybrid/reranking stack, and the complete LegalAI hybrid + reranking pipeline. For InLegalBERT, measure section-classifier micro/macro-F1 and NER entity-level F1 against the base/non-fine-tuned baseline.

Task 4.A.2 — AI Performance Profiling
Record internal timing for parsing, NER, classification, retrieval, reranking, and generation. Target under 15 seconds for normal text professional queries, use a broader target for digital PDFs, and treat OCR-heavy PDFs as best-effort with progress feedback. Optimise only after profiling identifies the dominant bottleneck.

Task 4.R.1 — Backend Hardening and Integration Tests (Roshinee)
Add consistent structured error responses and Pydantic validation across backend routes. Enforce maximum upload size, query-length limits, required ingestion metadata, invalid/expired auth handling, role errors, database failures, and LLM upstream failures. Add integration tests for auth, ingestion status, analytics endpoints, and query-history persistence.

Task 4.V.1 — Professional Frontend Error States (Ravindran)
Every Professional Mode API interaction must have loading, success, empty, and retryable error states. Handle unavailable AI service with a user-friendly message without exposing backend internals.

Task 4.G.1 — Public/Admin Error States (Gugan)
Add equivalent error handling for Public/Admin pages. Public messages must remain plain-language. Admin ingestion errors should show actionable per-file status without exposing sensitive stack traces.

Task 4.ALL — End-to-End Integration, Deployment and User Acceptance Testing
All four developers participate. Run vertical integration tests from browser to database/vector store, deploy the backend/frontend and persistent services, and conduct structured testing with at least 2–3 law students or legal practitioners for Professional Mode and 3–5 non-technical users for Public Mode. Collect feedback on source grounding, output usefulness, UI clarity, and reliability; implement critical fixes before final submission.

Task Dependencies and Build Order

Dependencies are classified as **BLOCKING**, **CONTRACT**, or **PARALLEL**. A BLOCKING dependency must be complete before the dependent work can function. A CONTRACT dependency means implementation may proceed against an agreed schema/mock and only live integration must wait. PARALLEL work is intentionally expected to proceed at the same time.

Task / Workstream	Dependency Type	Dependency and Build Note
Phase 1 start	BLOCKING	Step 0.1 repository structure + initial Step 0.2 environment/API-contract freeze
Step 0.3 DB schema	PARALLEL	Begins immediately after repo/environment setup; full dataset collection is not required
Step 0.4 datasets	PARALLEL	Runs alongside all Phase 1/2/3 implementation using a smaller development corpus first
Step 0.5 fine-tuning	PARALLEL	Final checkpoints are not required for wrapper/API development; base model/stubs may be used until integration
1.A.1 document/vector services	BLOCKING	Requires repo/config only; can proceed in parallel with DB/auth/frontend work
1.A.2 NLP services	CONTRACT	Wrapper can begin before final fine-tuned checkpoint; final model integration waits for Step 0.5
1.A.3 retrieval/reranking	BLOCKING	Requires vector interface and a small indexed corpus; does not require the full target corpus
1.A.4 LLM layer	BLOCKING	Requires API/config credentials only; can run in parallel with retrieval and NLP
1.A.5 query orchestration	BLOCKING	Requires stable parser/NLP/retrieval/LLM interfaces; each service may initially be stubbed for integration testing
1.R.1 backend foundation	BLOCKING	Requires repo/environment + DB design; independent of RAG completion
1.R.2 ingestion/jobs	CONTRACT	Can be implemented against Ananthu's parser/chunker/embedder/vector interfaces; live ingestion waits for concrete services
1.R.3 analytics/history	BLOCKING	Requires PostgreSQL models/data; independent of query-generation completion
2.V.1 professional foundation	CONTRACT	Begins after API/auth contracts are frozen; live backend is not required
2.V.2 login/guards	CONTRACT	UI can use mocked auth contract; live verification waits for 1.R.1
2.V.3 professional pages	CONTRACT	Build against `ProfessionalQueryResponse`; live integration waits for 1.A.5

Task / Workstream	Dependency Type	Dependency and Build Note
2.V.4 analytics dashboard	CONTRACT	Build charts against mock analytics JSON; live integration waits for 1.R.3
2.V.5 history UI	CONTRACT	Build against history schema; live integration waits for 1.R.3
3.G.1 shared UX	PARALLEL	Begins with Next.js setup and shared API contracts; no backend endpoint required
3.G.2 public UI	CONTRACT	Build against `PublicQueryResponse`; live integration waits for 1.A.5
3.G.3 admin UI	CONTRACT	Build against ingestion/status/corpus contracts; live integration waits for 1.R.2
Step 0.6 full corpus ingestion	BLOCKING	Requires stable ingestion pipeline; scale only after a small corpus passes integration checks
Phase 4 evaluation	BLOCKING	Requires live integrated vertical slices and held-out evaluation data

Recommended integration checkpoints:

1. **Checkpoint 1 — Foundation:** Next.js reaches FastAPI health endpoint; PostgreSQL connection works; protected login works.
2. **Checkpoint 2 — Retrieval slice:** one sample PDF parses, chunks, embeds, indexes, and can be retrieved.
3. **Checkpoint 3 — Professional slice:** text query reaches the complete RAG pipeline and renders in Professional Mode.
4. **Checkpoint 4 — Parallel product slices:** Public query renders in Public Mode; analytics data renders in the dashboard; Admin ingestion job status is visible.
5. **Checkpoint 5 — Phase 1 review build:** all milestone components run together with the representative corpus and real structured data.

Future Work

The following feature is intentionally deferred beyond the present Phase 1 implementation scope and will be discussed separately for Phase 2.

CourtSim — AI-Powered Mock Trial Simulator for Indian Criminal Proceedings

LegalAI provides the foundational services required for a CourtSim-style system: a grounded RAG knowledge base of Indian criminal judgments, IPC/BNS section identification, source-aware retrieval, structured case analysis, and reusable query/retrieval services. A later implementation can extend this foundation with role-specific Prosecutor, Defence Counsel, and Judge agents operating in a multi-turn simulation grounded in the same LegalAI corpus and Indian criminal procedure.

The Phase 1 implementation should therefore keep document processing, retrieval, reranking, and

generation behind reusable service interfaces rather than coupling them only to a single endpoint. CourtSim does not otherwise change the Phase 1 architecture and is not part of the Phase 1 delivery scope.

This is an internal development document for the LegalAI project team. It is not intended for external distribution.
Last updated: September 2, 2026